

Wati.io | CAB Engineer Candidate Assignment

Project: WATI WhatsApp Automation Agent

Objective

Build a lightweight AI Agent that allows a non-technical user to configure WhatsApp business workflows through natural language.

The agent bridges the gap between human intent and API execution:

"I want to do X when a customer does Y" → The agent figures out which WATI APIs to call, in what order, and executes (or previews) the plan.

1. Open-Ended Design Philosophy

This is an open-ended assignment with no single "correct" solution. We view this as a test of your system-thinking ability and technical trade-offs.

Founder Mindset: Treat this as if you were the founding engineer building V1 of an internal automation tool.

Creative Liberty: We encourage you to explore original ideas in agent behavior, tool orchestration, and error handling strategies.

Prioritization: A small, well-thought-out demo is vastly preferred over a large, incomplete one.

2. Technical & Product Requirements

A. Core Capability

The agent must be able to take a natural-language instruction and translate it into a concrete plan of WATI API calls. At minimum, the agent should handle instructions that combine two or more of the following API domains:

Domain	Example Actions
Contacts	Look up a contact, add/update contact attributes, manage tags
Messages	Send a text message, send a template message, send an interactive message
Templates	List available templates, look up template details
Broadcasts	Create and send a broadcast to a segment
Operators & Tickets	Assign a conversation to an operator/team, create or resolve a ticket

Example user instructions (for illustration, not a fixed test set):

"Find all contacts tagged 'VIP' and send them the 'renewal_reminder' template with their name filled in."

"When I say 'escalate 6281234567890', assign that contact's conversation to the Support team and add the tag 'escalated'."

"Send a broadcast with the 'flash_sale' template to all contacts who have the attribute 'city' = 'Jakarta'."

B. Implementation Specs

Language/Stack: Use any language or framework you are comfortable with. Python, TypeScript/Node, or Go are common choices.

LLM Integration: You must use at least one LLM (OpenAI, Anthropic, open-source, etc.) for intent understanding and/or plan generation. Clearly explain your choice.

Real API Calls (Preferred) or Realistic Mock: You may either call the WATI sandbox API (credentials provided separately) or implement a realistic mock layer that simulates API responses. If you mock, make the mock layer clearly swappable with real calls.

Interface: The primary interaction can be CLI, a chat-based web UI, or a messaging bot — your choice. Prioritize demonstrating the agent logic over UI polish.

C. What We're Looking For

Dimension	What Impresses Us
API Comprehension	You quickly grasped the relationships between WATI's API domains and made sensible orchestration choices
Agent Architecture	Clean separation of LLM reasoning, tool/API execution, and state management
Error Handling	Graceful handling of ambiguous input, missing parameters, API failures, and partial execution
Product Sense	Thoughtful UX choices — e.g., confirming destructive actions, previewing the plan before execution, explaining what each step does
Code Quality	Readable, well-structured code. No over-engineering, no under-engineering

D. Optional Extensions

Feel free to extend the demo in any direction that showcases your strengths:

Multi-step planning with rollback — If step 3 of 5 fails, what happens?

Conversational memory — The agent remembers context from previous instructions in the same session.

Dry-run / preview mode — Show the user exactly what API calls will be made before executing.

Parameter validation — Proactively ask the user for missing info instead of failing silently.

Batch awareness — Handle large operations intelligently (rate limits, pagination, progress reporting).

3. WATI API Reference

We provide a simplified API reference below. This is a subset of the real WATI API — enough for the assignment.

Base URL: https://live-mt-server.wati.io/{YOUR_TENANT_ID}

Auth: Authorization: Bearer {TOKEN} (provided separately)

Contacts

GET /api/v1/getContacts?pageSize=20&pageNumber=1

GET /api/v1/getContacts?tag=VIP

GET /api/v1/getContactInfo/{whatsappNumber}

POST /api/v1/addContact/{whatsappNumber}

Body: { "name": "...", "customParams": [{"name": "city", "value": "Jakarta"}] }

POST /api/v1/updateContactAttributes/{whatsappNumber}

Body: { "customParams": [{"name": "...", "value": "..."}] }

Tags

POST /api/v1/addTag/{whatsappNumber} Body: { "tag": "VIP" }

DELETE /api/v1/removeTag/{whatsappNumber}/{tagName}

Messages

POST /api/v1/sendSessionMessage/{whatsappNumber}

Body: { "messageText": "Hello!" }

POST /api/v1/sendTemplateMessage/{whatsappNumber}

Body: { "template_name": "...", "broadcast_name": "...", "parameters": [{"name": "1", "value": "..."}] }

POST /api/v2/sendTemplateMessage/{whatsappNumber}

Body: {

"template_name": "...", "broadcast_name": "...", "parameters": [{"name": "body_1", "value": "..."}] }

Templates

GET /api/v1/getMessageTemplates?pageSize=20&pageNumber=1

Broadcasts

POST /api/v1/sendBroadcastToSegment

Body: { "template_name": "...", "broadcast_name": "...", "segmentName": "..." }

Operators & Tickets

GET /api/v1/getOperators

POST /api/v1/assignOperator/{whatsappNumber} Body: { "email": "agent@company.com" }

POST /api/v1/tickets/assign Body: { "whatsappNumber": "...", "teamName": "Support" }

Note: You do NOT need to use every endpoint. Choose the subset that best supports your demo scenario.

4. Constraints & Expectations

Timebox: Please spend approximately 3 hours. We respect your time — do not over-invest.

Quality over Quantity: Focus on demonstrating how you think, prioritize, and architect, rather than covering every API endpoint.

Data: Mock data is perfectly acceptable where needed. No production-grade backend is required.

LLM Costs: Use whichever model you prefer. If cost is a concern, smaller/cheaper models are fine — we care about the architecture, not the model size.

5. Deliverables

Please submit the following:

Working Demo: A GitHub repository (public or private with access granted) containing the complete project with a clear README on how to run it.

Demo Recording: A short video (3–5 minutes) showing the end-to-end experience — give the agent a few natural-language instructions and walk us through what happens.

Short Write-up (can be in the README or a separate doc) covering:

Problem Framing: How you interpreted the assignment and scoped your MVP.

Architecture: How the agent works — LLM prompt design, tool/API mapping strategy, execution flow.

AI/LLM Usage: How LLMs were utilized and why you made those choices.

Build Notes (Required): A breakdown of how you spent your time, what you prioritized, what you intentionally did not build, and what the "V2" roadmap would look like.

Trade-offs: Any interesting decisions you made and why.

6. Evaluation Criteria

Weight	Criteria	Description
30%	Agent Design & Architecture	Is the agent well-structured? Is the LLM-to-API mapping clean and extensible?
25%	API Understanding & Integration	Did the candidate correctly understand the API semantics and relationships?
20%	Product Thinking & UX	Does the agent feel usable? Are edge cases handled gracefully?
15%	Code Quality	Is the code readable, well-organized, and appropriately abstracted?
10%	Write-up & Communication	Is the reasoning clear? Are trade-offs well-articulated?

Best regards, The Wati Team